LeccionRegresionLogistica

November 4, 2026

[6]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from sklearn import metrics
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")

import os
```

[7]:
```
col_names = ["pregnant", "glucose", "bp", "skin", "insulin", "bmi", "pedigre",␣
 ↪"age", "label"]
pima = pd.read_csv("diabetes.csv", header=None, names = col_names)
df = pima.iloc[1:, :]
df
```

[7]:

	pregnant	glucose	bp	skin	insulin	bmi	pedigre	age	label
1	6	148	72	35	0	33.6	0.627	50	1
2	1	85	66	29	0	26.6	0.351	31	0
3	8	183	64	0	0	23.3	0.672	32	1
4	1	89	66	23	94	28.1	0.167	21	0
5	0	137	40	35	168	43.1	2.288	33	1
..	…	…	..	…	…	…	…	..	…
764	10	101	76	48	180	32.9	0.171	63	0
765	2	122	70	27	0	36.8	0.34	27	0
766	5	121	72	23	112	26.2	0.245	30	0
767	1	126	60	0	0	30.1	0.349	47	1
768	1	93	70	31	0	30.4	0.315	23	0

[768 rows x 9 columns]

[8]:
```
# Selección de las caracteristicas
feature_cols = ["pregnant", "insulin", "bmi", "age", "glucose", "bp", "pedigre"]
X = df[feature_cols]
y = df.label
```

[9]:
```
# Creación de las bases de entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,␣
↪random_state = 1)
```

[10]:
```
# sag: Stochastic Average Gradient
# newton-cg: Newton
# liblinear: Library for large linear classification (Algoritmo Coordiante␣
↪Descent)
# saga: Variante del método sag
# lbfgs: Limited-memory Broyden-Fletcher-Goldfarb-Shanno Algorithm
```

#Regresión logística con Solucionador (Solver): sag

[11]:
```
model = LogisticRegression(solver="sag")
clf = model.fit(X_train, y_train)

# Predicción de etiquetas de clase sobre datos de prueba
y_pred = model.predict(X_test)

# Impresión de coeficientes de la regresión de puntajes
print("Intercepto (Beta 0)", clf.intercept_)
print("Pesos de cada variable (Beta1, Beta2, ..., Beta7)", clf.coef_)
```

```
Intercepto (Beta 0) [-0.01186487]
Pesos de cada variable (Beta1, Beta2, …, Beta7) [[ 0.05230694  0.00116428
-0.00937608 -0.0023585   0.01408102 -0.03084549
   0.00106297]]
```

[12]:
```
# Evaluación de la precisión del modelo
score = model.score(X_test, y_test)
print("Score de precisión: ", score)
```

```
Score de precisión:  0.6428571428571429
```

[13]:
```
# Reporte de clasificación
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.67      0.89      0.76        99
           1       0.50      0.20      0.29        55

    accuracy                           0.64       154
   macro avg       0.58      0.54      0.52       154
weighted avg       0.61      0.64      0.59       154
```

#REGRESIÓN LOGÍSTICA CON SOLUCIONADOR (SOLVER): NEWTON-CG

[16]:
```
model = LogisticRegression(solver="newton-cg")
clf = model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Intercepto (Beta0): ", clf.intercept_)
print("Pesos (Betas): ", clf.coef_)
print("------------------------------------------------------------")
score = model.score(X_test, y_test)
print("Precisión global: ", score)
print("------------------------------------------------")
print(classification_report(y_test, y_pred))
```

```
Intercepto (Beta0):  [-8.10390151]
Pesos (Betas):  [[ 0.12718791 -0.00096255  0.08559354  0.01168397  0.03656972
-0.01508705
   0.6336633 ]]
------------------------------------------------------------
Precisión global:  0.7727272727272727
------------------------------------------------
              precision    recall  f1-score   support

           0       0.78      0.90      0.84        99
           1       0.75      0.55      0.63        55

    accuracy                           0.77       154
   macro avg       0.77      0.72      0.73       154
weighted avg       0.77      0.77      0.76       154
```

#Regresión logística con Solucionador (Solver): liblinear

[17]:
```
model = LogisticRegression(solver="liblinear")
clf = model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Intercepto (Beta0): ", clf.intercept_)
print("Pesos (Betas): ", clf.coef_)
print("------------------------------------------------------------")
score = model.score(X_test, y_test)
print("Precisión global: ", score)
print("------------------------------------------------")
print(classification_report(y_test, y_pred))
```

```
Intercepto (Beta0):  [-5.38033815]
Pesos (Betas):  [[ 1.18843413e-01 -3.17269639e-04  5.28290919e-02
3.80596808e-03
```

```
   2.87936243e-02 -1.90988005e-02  4.74903576e-01]]
------------------------------------------------------------
Precisión global:  0.7662337662337663
------------------------------------------------
              precision    recall  f1-score   support

           0       0.77      0.90      0.83        99
           1       0.74      0.53      0.62        55

    accuracy                           0.77       154
   macro avg       0.76      0.71      0.72       154
weighted avg       0.76      0.77      0.76       154
```

#Regresión logística con Solucionador (Solver): saga

[18]:
```
model = LogisticRegression(solver="saga")
clf = model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Intercepto (Beta0): ", clf.intercept_)
print("Pesos (Betas): ", clf.coef_)
print("------------------------------------------------------------")
score = model.score(X_test, y_test)
print("Precisión global: ", score)
print("------------------------------------------------")
print(classification_report(y_test, y_pred))
```

```
Intercepto (Beta0):  [-0.0060933]
Pesos (Betas):  [[ 0.02995418  0.00115522 -0.00951769  0.00094955  0.01389563
-0.03081644
   0.00045569]]
------------------------------------------------------------
Precisión global:  0.6493506493506493
------------------------------------------------
              precision    recall  f1-score   support

           0       0.67      0.89      0.77        99
           1       0.52      0.22      0.31        55

    accuracy                           0.65       154
   macro avg       0.60      0.55      0.54       154
weighted avg       0.62      0.65      0.60       154
```

#Regresión logística con Solucionador (Solver):lbfgs

[19]:
```
model = LogisticRegression(solver="lbfgs")
clf = model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Intercepto (Beta0): ", clf.intercept_)
print("Pesos (Betas): ", clf.coef_)
print("------------------------------------------------------------")
score = model.score(X_test, y_test)
print("Precisión global: ", score)
print("------------------------------------------------")
print(classification_report(y_test, y_pred))
```

```
Intercepto (Beta0):  [-8.0970774]
Pesos (Betas):  [[ 0.12707249 -0.00096077  0.08549919  0.01168769  0.03655047
-0.01509716
   0.63316019]]
------------------------------------------------------------
Precisión global:  0.7727272727272727
------------------------------------------------
              precision    recall  f1-score   support

           0       0.78      0.90      0.84        99
           1       0.75      0.55      0.63        55

    accuracy                           0.77       154
   macro avg       0.77      0.72      0.73       154
weighted avg       0.77      0.77      0.76       154
```

[21]:
```
# Predicción de probabilidades para casos particulares
y_pred_probs = clf.predict_proba(X_test)[:,1]
y_pred_probs[0:5]
```

[21]:
```
array([0.4366193 , 0.31979373, 0.15802561, 0.04345572, 0.1997421 ])
```

[22]:
```
y_test[0:5]
```

[22]:
```
286    0
102    0
582    0
353    0
727    0
Name: label, dtype: object
```

[23]:
```
y_test = y_test.to_numpy()
y_test[0:5]
```

```
y_test = [int(numeric_string) for numeric_string in y_test]
y_test[0:5]
```

[23]:
```
[0, 0, 0, 0, 0]
```

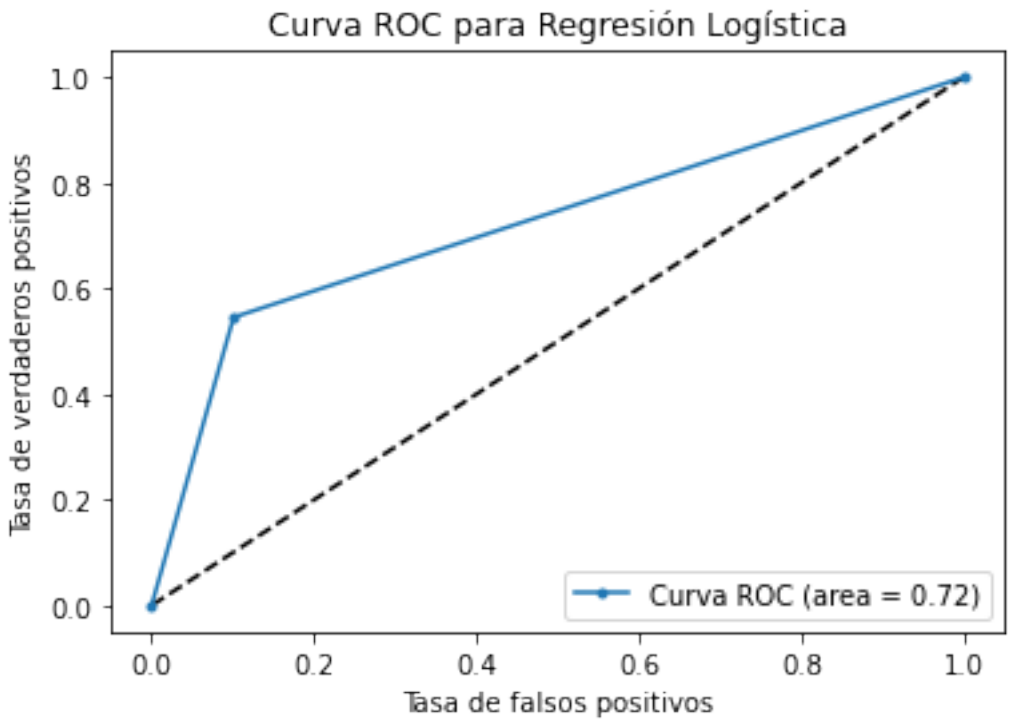
[24]:
```
import numpy as np
y_pred_probs = np.round(y_pred_probs,0)
y_pred_probs[0:5]
```

[24]:
```
array([0., 0., 0., 0., 0.])
```

[25]:
```
# Graficación de la curva ROC y cálculo del área bajo ella
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
# fpr = false positive rate
# tpr = true positive rate
# thresholds = límites para valores de p
fpr, tpr, thresholds = roc_curve(y_test, y_pred_probs)
auc = roc_auc_score(y_test, y_pred_probs)
```

[26]:
```
plt.plot([0,1], [0,1], 'k--')
plt.plot(fpr, tpr, marker='.', label = "Curva ROC (area = %0.2f)" % auc)
plt.xlabel("Tasa de falsos positivos")     # False positive rate
plt.ylabel("Tasa de verdaderos positivos") # True positive rate
plt.legend(loc = "lower right")
plt.title("Curva ROC para Regresión Logística")
```

[26]:
```
Text(0.5, 1.0, 'Curva ROC para Regresión Logística')
```


Curva ROC para Regresión Logística
Tasa de verdaderos positivos
Tasa de falsos positivos
0.0
0.2
0.4
0.6
0.8
1.0
Curva ROC (area = 0.72)

[ ]: